

JMH



Benchmarking Intro



- Mitunter sind einige Teile der Software nicht so performant, wie benötigt.
- Zur Optimierung der Performance gibt es verschiedene Hilfsmittel und Ebenen
- Generell sollte man zunächst gründlich messen und nur mit Bedacht optimieren.
- Wieso?
  - bereits diverse Optimierungen in die JVM eingebaut
  - nicht trivial, da die Messungen unter möglichst gleichen Bedingungen (CPU-Last, Speicherverbrauch usw.) geschehen sollten, für vergleichbare Resultate
  - rein auf Basis von Vermutungen liegt man häufig falsch
- keinesfalls nur aufgrund von Vermutungen, sondern basierend auf Messungen:
  - einfache Start-/Stop-Messungen
  - empfehlenswerter sind ausgeklügeltere Verfahren mit mehreren Durchläufen

Einfache Start-/Stop-Messungen (Bitte nicht machen!!)



- Einfache Start-/Stop-Messungen mit `System.currentTimeMillis()`

```
// ACHTUNG: keine gute Variante
final long startTimeMs = System.currentTimeMillis();

someCodeToMeasure();

final long stopTimeMs = System.currentTimeMillis();
final long duration = stopTimeMs - startTimeMs;
```

- den zu messenden Programmteil mit `System.currentTimeMillis()` umklammern
- Als Art Stoppuhr nutzen, indem man die Differenz zwischen den Werten ermittelt
- Genauer – aber nicht wirklich besser – wird es mit `System.nanoTime()`

Wiederholte Start-/Stop-Messungen (Möglichst vermeiden!)



- Wiederholte Start-/Stop-Messungen und besserer (genauerer) Timer

```
// bessere Variante
final long startTimeNs = java.lang.System.nanoTime();

for (int i = 0; i < MAX_ITERATIONS; i++)
{
   someCodeToMeasure();
}

final long stopTimeNs = java.lang.System.nanoTime();

final long avgDurationNs = (stopTimeNs - startTimeNs) / MAX_ITERATIONS;
```

- Durch mehrere Durchläufe und Durchschnittsbildung weniger anfällig für Systemlastschwankungen oder sonstige Störeinflüsse.
- recht einfach auch Minimal- und Maximaldauer oder Standardabweichung zu ermitteln.

Wiederholte Start-/Stop-Messungen mit Warm-up (Wird kompliziert)



- Einschwing-Effekte: Erst nach einer gewissen Anzahl an Durchläufen zeigt eine Funktionalität ihre optimale Laufzeit:

```
// noch bessere Variante der Messung durch Warm-up
for (int i = 0; i < MAX_WARMUP_ITERATIONS; i++)
{
   someCodeToMeasure();
}

// eigentliche Messung nach Warm-up
final long startTimeNs = java.lang.System.nanoTime();

for (int i = 0; i < MAX_ITERATIONS; i++)
{
   someCodeToMeasure();
}
final long stopTimeNs = java.lang.System.nanoTime();

final long avgDurationNs = (stopTimeNs - startTimeNs) / MAX_ITERATIONS;
```

JEP 230: Micobenchmark Suite / JMH als Unterstützung



- JEP 230 goal: add a basic suite of microbenchmarks to the JDK source code, and make it easy for developers to run existing microbenchmarks and create new ones.
- Basiert auf Java Microbenchmark Harness (JMH)
- Framework zum Erstellen von Microbenchmark-Tests
- Microbenchmarking = Optimierungsebene einzelner bzw. weniger Anweisungen
- Berücksichtigt verschiedene externe Störeinflüsse und Schwankungen
- Umfangreich, aber meistens einfach konfigurierbar
- Macht das Schreiben von Benchmarks fast so einfach wie Unit Testing mit JUnit

Microbenchmarks mit JMH



- Eine Performance-Test-Umgebung kann JMH mit folgendem Maven-Kommando erzeugen:

```
mvn archetype:generate \
 -DinteractiveMode=false \
 -DarchetypeGroupId=org.openjdk.jmh \
 -DarchetypeArtifactId=jmh-java-benchmark-archetype \
 -DgroupId=org.sample \
 -DartifactId=jmh-test \
 -Dversion=1.0-SNAPSHOT
```

Microbenchmarks mit JMH



- Als Grundgerüst wird eine Klasse `MyBenchmark` erzeugt:

```
public class MyBenchmark
{
    @Benchmark
    public void testMethod()
    {
        // This is a demo/sample template for building your JMH benchmarks. Edit
            as needed.
        // Put your benchmark code here.
    }
}
```

- JMH arbeitet mit Annotations und integriert basierend darauf verschiedene Messungen.

Microbenchmarks mit JMH



- **Basierend auf dem Grundgerüst kann man eigene Benchmark-Klassen erstellen:**

```
├── pom.xml
├── src
│   └── main
│       └── java
│           └── org
│               └── sample
│                   ├── MyBenchmark.java
│                   ├── FirstIntegerBenchmark.java
│                   └── SimpleMathBenchmark.java
└── target
    ├── benchmarks.jar
```

- **Mit 2 Schritten zum Benchmark**

```
1) mvn clean package
2) java -jar target/benchmarks.jar
```

Eigener Microbenchmark mit JMH



```
@Measurement(iterations = 3, time = 1000, timeUnit = TimeUnit.MILLISECONDS)
@Warmup(iterations = 7, time = 1000, timeUnit = TimeUnit.MICROSECONDS)
@Fork(4)
public class FirstIntegerBenchmark
{
	@Benchmark
	public String numberAsHexString()
	{
		int dec = 123456789;
		return Integer.toHexString(dec);
	}

	@Benchmark
	public String numberAsBinaryString()
	{
		int dec = 123456789;
		return Integer.toBinaryString(dec);
	}
}
```

Eigene Microbenchmarks mit JMH



```
// Based on https://www.retit.de/continuous-benchmarking-with-jmh-and-junit-2/
public class SearchBenchmark {
    @State(Scope.Thread)
    public static class SearchState {
        public String text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ___abcdefghijklmnopqrstuvwxyz";
    }

    @Benchmark
    public int testIndexOf(SearchState state) {
        return state.text.indexOf("M");
    }

    @Benchmark
    public int testIndexOfChar(SearchState state) {
        return state.text.indexOf('M’);
    }

    @Benchmark
    public boolean testContains(SearchState state) {
        return state.text.contains("M");
    }
}
```

Eigene Microbenchmarks mit JMH



```
@BenchmarkMode(Mode.AverageTime)
@Fork(2)
@State(Scope.Benchmark)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
public class SimpleStringJoinBenchmark
{
    private String from = "Michael";
    private String to = "Participants";
    private String subject = "Benchmarking with JMH";

    @Benchmark
    public String stringPlus(Blackhole blackhole)
    {
        String result = "From: " + from + "\nTo: " + to + "\nSubject: " + subject;
        blackhole.consume(result);
        return result;
    }
```

Inspiriert von http://alblue.bandlem.com/2016/04/jmh-stringbuffer-stringbuilder.html, aber hier mit Blackhole und leicht abgewandelt

Eigene Microbenchmarks mit JMH



```
@Benchmark
public String stringPlusEqual(Blackhole blackhole)
{
    String result = "From: " + from;
    result += "\nTo: " + to;
    result += "\nSubject: " + subject;

    blackhole.consume(result);
    return result;
}

@Benchmark
public String builderAppendChained(Blackhole blackhole)
{
    String result = new StringBuilder().append("From: ").append(from).
                                        append("\nTo: ").append(to).
                                        append("\nSubject: ").append(subject).
                                        toString();
    blackhole.consume(result);
    return result;
}
```

ACHTUNG – Eigene Microbenchmarks mit JMH



```
@State(Scope.Benchmark)
public static class MyBenchmarkState {
    @Param({ "10000", "100000" })
    public int value;
}


@Benchmark
public String stringPlusABC(MyBenchmarkState state, Blackhole blackhole) {
    String result = "";
    for (int i = 0; i < state.value; i++) {
        result += "ABC";
    }

    blackhole.consume(result);
    return result;
}
```

ACHTUNG – Eigene Microbenchmarks mit JMH



```
@Benchmark
public String stringConcatABC(MyBenchmarkState state, Blackhole blackhole) {
    String result = "";
    for (int i = 0; i < state.value; i++) {
        result = result.concat("ABC");
    }
    blackhole.consume(result);
    return result;
}

@Benchmark
public String concatUsingStringBuilder(MyBenchmarkState state, Blackhole blackhole) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < state.value; i++) {
        sb.append("ABC");
    }
    String result = sb.toString();
    blackhole.consume(result);
    return result;
}
```

Microbenchmarks mit JMH



- **POM anpassen:**

```
<!-- Java source/target to use for compilation. -->
        <javac.target>1.8</javac.target>
=>
<javac.target>17</javac.target>

<groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.0</version>
=>
<version>3.8.1</version>
```



Was wollen wir messen?

- `indexOf(String), indexOf(char), contains(String)`
- `for, forEach, while, Iterator`
- `String +=, Sting.concat(), StringBuilder.append()`

Beispielergebnisse



Benchmark	Mode	Cnt	Score	Error	Units
LoopBenchmark.loopFor	avgt	10	5.039	± 0.134	ms/op
LoopBenchmark.loopForEach	avgt	10	5.308	± 0.322	ms/op
LoopBenchmark.loopIterator	avgt	10	5.466	± 0.528	ms/op
LoopBenchmark.loopWhile	avgt	10	5.026	± 0.218	ms/op

Benchmark	Mode	Cnt	Score	Error	Units
SearchBenchmark.testContains	avgt	15	7.712	± 0.241	ns/op
SearchBenchmark.testIndexOf	avgt	15	7.797	± 0.475	ns/op
SearchBenchmark.testIndexOfChar	avgt	15	7.046	± 0.070	ns/op

Beispielergebnisse



Benchmark	Mode	Cnt	Score	Error	Units
SimpleStringJoinBenchmark.builderAppendChained	avgt	10	46.308	± 7.950	ns/op
SimpleStringJoinBenchmark.builderMultipleAppend	avgt	10	127.312	± 14.935	ns/op
SimpleStringJoinBenchmark.stringConcat	avgt	10	83.888	± 18.979	ns/op
SimpleStringJoinBenchmark.stringPlus	avgt	10	38.871	± 2.823	ns/op
SimpleStringJoinBenchmark.stringPlusEqual	avgt	10	39.346	± 3.015	ns/op

Beispielergebnisse



Benchmark	Mode	Cnt	Score	Error	Units
SimpleStringJoinBenchmark.builderAppendChained	avgt	10	46.308	± 7.950	ns/op
SimpleStringJoinBenchmark.builderMultipleAppend	avgt	10	127.312	± 14.935	ns/op
SimpleStringJoinBenchmark.stringConcat	avgt	10	83.888	± 18.979	ns/op
SimpleStringJoinBenchmark.stringPlus	avgt	10	38.871	± 2.823	ns/op
SimpleStringJoinBenchmark.stringPlusEqual	avgt	10	39.346	± 3.015	ns/op

Benchmark	(value)	Mode	Cnt	Score	Error	Units
StringJoinBenchmark.concatUsingStringBuilder	10000	avgt	10	0.016	± 0.001	ms/op
StringJoinBenchmark.concatUsingStringBuilder	100000	avgt	10	0.172	± 0.009	ms/op
StringJoinBenchmark.stringConcatABC	10000	avgt	10	9.634	± 0.812	ms/op
StringJoinBenchmark.stringConcatABC	100000	avgt	10	662.953	± 4.029	ms/op
StringJoinBenchmark.stringPlusABC	10000	avgt	10	7.926	± 0.149	ms/op
StringJoinBenchmark.stringPlusABC	100000	avgt	10	662.343	± 2.912	ms/op